

รายงานโครงงาน

รายวิชา CP413008 Machine Learning Engineering for Production

วิศวกรรมการเรียนรู้ของเครื่องสำหรับการผลิต

Github link: <https://github.com/super-SuPer-dev/loan-mlops>

ระบบคัดกรองคำขอสินเชื่อ

(Loan Approval MLOps)

กลุ่มที่ 7

สมาชิก	รหัสนักศึกษา	บัญชี GitHub	งานหลัก (PR)
นายณภัทร โสตาตง	673380076-1	naphat67	สคริปต์รัน test case ของอาจารย์ (#12)
นายพุฒิพงศ์ พานิชพันธุ์	673380335-3	super-SuPer-dev	แก้ cascade และทดสอบ latency (#9), Rollback DAG (#10), ความพร้อมวันสาธิต (#11)
นายจักรภัทร เอี่ยมเสริม	673380309-4	jakrapatiam-create	รายงาน drift ด้วย Evidently (#14)
นายอภิมุข เทียสักษา	673380354-9	Tammy-AP	ทดสอบ monitoring และ coverage ใน CI (#13)
นางสาวเดชนิ ถามูลตรี	673380358-1	techineethamoontree	รายงานฉบับส่งและสไลด์ (#15)

เสนอ อ. ดร.วันเฉลิม นัตตา

ภาคเรียนที่ 1 ปีการศึกษา 2569

สารบัญ

สรุปผลงาน	4
1. การวางกรอบปัญหา	4
1.1 โจทย์และเหตุผลที่ใช้ Machine Learning	4
1.2 AI Project Canvas	4
2. ตัวชี้วัด	6
2.1 Optimizing metric และ Gating metric	6
2.2 ตัวชี้วัดทางธุรกิจ	6
2.3 คำตอบคำถามตรวจสอบหัวข้อ	6
3. สถาปัตยกรรมระบบ	7
3.1 แผนภาพสถาปัตยกรรม	7
3.2 เหตุผลการเลือกเครื่องมือ	8
3.3 เวอร์ชันไลบรารีหลัก	8
4. ข้อมูลและการตรวจสอบคุณภาพ	9
4.1 ภาพรวมชุดข้อมูล	9
4.2 คอลัมน์ที่ไม่ใช่เป็นฟีเจอร์	10
4.3 การแบ่งข้อมูล	10
4.4 Schema และการตรวจความผิดปกติ	11
4.5 การแปลงข้อมูลและการกัน Training-Serving Skew	11
5. การพัฒนาโมเดล	11
5.1 การทดลองและการเลือกโมเดล	11
5.2 Threshold ตามต้นทุน	12
5.3 การอธิบายผล	12
5.4 Fairness	12
6. Experiment Tracking และ Model Registry	13
6.1 สิ่งที่น่าสนใจในทุกการทดลอง	13
6.2 ทะเบียนโมเดลและด้านตรวจก่อนอนุมัติ	15
6.3 Rollback	15
7. การให้บริการและ SLO	16
7.1 Endpoint	16
7.2 รูปแบบการให้บริการ: real-time + cascade	17

7.3 SLO	17
7.4 ผลวัดจริง	18
7.5 Logs	18
8. Monitoring และนโยบาย Retrain	18
8.1 การแยก Data Drift กับ Concept Drift	19
8.2 ผลสาธิต	19
8.3 นโยบาย Retrain	19
8.4 รายงาน Evidently	20
8.5 Prometheus และ Grafana	20
9. CI/CD, Pipeline และการทำซ้ำ	23
9.1 CI/CD	23
9.2 Pipeline แบบ DAG	24
9.3 การทำซ้ำและการรันจากเครื่องเปล่า	25
9.4 Git และการทำงานเป็นทีม	27
9.5 เทสต์และ coverage	27
9.6 สคริปต์ test case ของอาจารย์	28
10. การใช้ AI ช่วยงาน ข้อจำกัด และงานในอนาคต	28
10.1 การใช้ AI ช่วยงาน	28
10.2 ข้อจำกัด	28
10.3 งานในอนาคต	29
เอกสารอ้างอิง	29

สรุปผลงาน

ระบบคัดกรองคำขอสินเชื่อทำงานได้ครบวงจร ตั้งแต่ข้อมูลดิบจนถึงการให้บริการ และส่งรันทั้งหมดได้ด้วยปั๊มเดียวใน Airflow โมเดลได้ ROC-AUC 0.899 บนชุดทดสอบ ส่วน API มี p95 latency 66.7 ms ที่โหลด 20 คำขอ/วินาที ผ่าน SLO 200 ms

- **Repository:** github.com/super-SuPer-dev/loan-mlops ใช้ branch และ Pull Request และมี CI 5 job
- **ชุดข้อมูล:** Loan Approval Dataset 2026 (Kaggle, สัญญาอนุญาต MIT) 30,000 คำขอ 36 คอลัมน์
- **สิ่งที่สำคัญได้จริง:** ข้อมูลเสียทำให้ pipeline หยุดพร้อมแจ้งเตือน, ตรวจพบ data drift และ concept drift แยกกัน, retrain อัตโนมัติ และ rollback ด้วยคำสั่งเดียวหรือปุ่มเดียว

1. การวางกรอบปัญหา

1.1 โจทย์และเหตุผลที่ใช้ Machine Learning

ระบบทำนายว่าคำขอสินเชื่อควรได้รับการอนุมัติหรือไม่ แล้วตัดสินใจอัตโนมัติเฉพาะคำขอที่โมเดลมั่นใจ (ประมาณ 78% ของคำขอ) ส่วนที่เหลือส่งให้เจ้าหน้าที่สินเชื่อพิจารณา

ทำไมใช้ ML แทนการเขียนกฎ: การตัดสินใจขึ้นกับข้อมูล 27 คอลัมน์ที่ส่งผลรวมกัน เช่น คะแนนเครดิต ประวัติผิดนัด และอัตราส่วนหนี้ต่อรายได้ คะแนนเครดิตตัวเดียวแยกคำขอได้แค่ ROC-AUC 0.77 แต่โมเดลที่เรียนจากทุกฟีเจอร์ได้ 0.899 นอกจากนี้ เมื่อนโยบายธนาคารเปลี่ยน ระบบเทรนโมเดลใหม่จากข้อมูลล่าสุดได้เอง ไม่ต้องไล่แก้กฎทีละข้อ

สิ่งที่โมเดลเรียนรู้: label (Loan_Approved) คือผลการตัดสินใจของธนาคารในอดีต ไม่ใช่ผลการชำระหนี้จริง โมเดลจึงเรียนแบบการตัดสินใจของเจ้าหน้าที่ ซึ่งรวมถึงอคติที่อาจมีอยู่เดิม เราจึงตรวจ fairness ทุกครั้งที่เทรน (หัวข้อ 5.4)

1.2 AI Project Canvas

ตารางที่ 1 AI Project Canvas

องค์ประกอบ	รายละเอียด
Data · ข้อมูล	• ประวัติคำขอสินเชื่อ 30,000 รายการ • 29 features: ตัวเลข 17, หมวดหมู่ 10, อัตราส่วนที่สร้างเอง 2 • ตัด leakage และข้อมูลอ่อนไหวออก • ผลอนุมัติใหม่ เข้ามาต่อเนื่อง
Skills · ทักษะที่ต้องใช้	• Machine Learning (binary classification) • MLOps: เทรนใหม่อัตโนมัติ และ deploy เป็น API • Monitoring (Prometheus)
Output · ตัวชี้วัด	• ทำนาย Loan_Approved (Approved / Rejected) • วัดผลด้วย AUC และต้นทุนผิดพลาด 3:1

องค์ประกอบ	รายละเอียด
Value Proposition	● ลดเวลาพิจารณา จากหลายวัน เหลือไม่กี่วินาที (สำหรับคำขอที่ชัดเจน) ● เจ้าหน้าที่มีเวลา ดูเคสที่ค้างมากขึ้น ● ตัดสินใจ 3 แบบ ตามความมั่นใจ: อนุมัติอัตโนมัติ / ปฏิเสธอัตโนมัติ / ส่งให้เจ้าหน้าที่
Integration · ใช้งาน	● Real-time API: /predict (แคนดอร์ / แอป) ● Batch API: คำขอค้าง ● Retrain ทุกสัปดาห์ และเมื่อเกิด drift
Stakeholders	● เจ้าหน้าที่สินเชื่อ ● ฝ่ายบริหารความเสี่ยง ● ฝ่ายกำกับดูแล (ดูแลความเป็นธรรม)
Customers · ผู้ใช้	● ผู้ยื่นกู้: ได้ผลเร็วขึ้น ไม่ต้องรอหลายวัน ● เจ้าหน้าที่สินเชื่อ: ไขผลทำนาย ช่วยตัดสินใจ
Cost · ต้นทุน	● อนุมัติผิด (เกิดหนี้เสีย) แพงกว่าปฏิเสธผิด 3 เท่า (ต้นทุนความผิดพลาด 3:1) ● เทรนใหม่ทุกสัปดาห์: 3 โมเดลขนานกัน ใช้เวลาไม่ถึง 1 นาที ● ตรวจ drift: data drift (KS/Chi²) และ concept drift (AUC ตก) ● ติดตาม latency/error (Prometheus) และสัดส่วนการปฏิเสธแยกกลุ่ม
Revenue · ผลตอบแทน	● ลดต้นทุนความผิดพลาด 23% (1,879 → 1,446) เทียบกับ threshold 0.5 ● ประหยัดเวลาเจ้าหน้าที่: คำขอที่ชัดเจนไม่ต้องรอคนพิจารณา ● ลูกค้าได้ผลเร็ว ลดโอกาสเสียลูกค้าให้คู่แข่ง

2. ตัวชี้วัด

2.1 Optimizing metric และ Gating metric

Optimizing metric คือ ROC-AUC บน validation ส่วน gating metric มี 6 ข้อ โมเดลต้องผ่านทุกข้อจึงจะใช้งานได้ เกณฑ์ทุกตัวเก็บไว้ทีเดียวใน src/config.py

ตารางที่ 2 Optimizing metric และ Gating metric

ประเภท	ตัวชี้วัด	เกณฑ์	เหตุผล
Optimizing	ROC-AUC (validation)	สูงสุด	วัดความสามารถในการเรียงความเสี่ยง ไม่ขึ้นกับ threshold และคลาสค่อนข้างสมดุล (58 / 42)
Gating	ROC-AUC (test)	≥ 0.85	ถ้าต่ำกว่านี้ ค่าขอที่ตัดสินใจอัตโนมัติได้นั้นจะน้อยจนไม่คุ้ม
Gating	Recall คลาสปฏิเสธ	≥ 0.80	กั้นการอนุมัติผิดคน ซึ่งเสียหายมากกว่า
Gating	เทียบกับ champion เดิม	ต่ำกว่าไม่เกิน 0.003	กั้นการสลับไปใช้โมเดลที่แย่ลง
Gating	p95 latency ต่อ 1 คำขอ	≤ 25 ms	มาจาก SLO 200 ms ที่ 20 คำขอ/วินาที: 20×25 ms = 0.5 วินาทีต่อวินาที ให้ API ไม่ยุ่งเกิน 50%
Gating	ขนาดไฟล์โมเดล	≤ 50 MB	โหลดได้เร็วตอน reload หรือ rollback
Gating	ส่วนต่างอัตราอนุมัติระหว่างเพศ	≤ 0.10	ความเป็นธรรม (ตรวจใน CI)

2.2 ตัวชี้วัดทางธุรกิจ

ตัวชี้วัดทางธุรกิจเชื่อมกับโมเดลผ่าน threshold และโซนก้ำกึ่ง

- สัดส่วนคำขอที่ตัดสินใจอัตโนมัติ (ภาระงานเจ้าหน้าที่ที่ลดลง): 78.5% โดยตัดสินใจ 80.3% ในส่วนนี้
- ความแม่นยำแยกโซน: AUTO_APPROVE 93.2% / AUTO_REJECT 74.9% โซน AUTO_APPROVE ควรแม่นยำที่สุด เพราะอนุมัติผิดมีต้นทุนสูงสุด
- ต้นทุนความผิดพลาดรวม (อนุมัติผิด 3 : ปฏิเสธผิด 1) ใช้เลือก threshold
- เวลาตอบกลับลูกค้า: $p95 \leq 200$ ms

2.3 คำตอบคำถามตรวจสอบหัวข้อ

ตารางที่ 3 คำตอบคำถามสำหรับตรวจสอบหัวข้อ

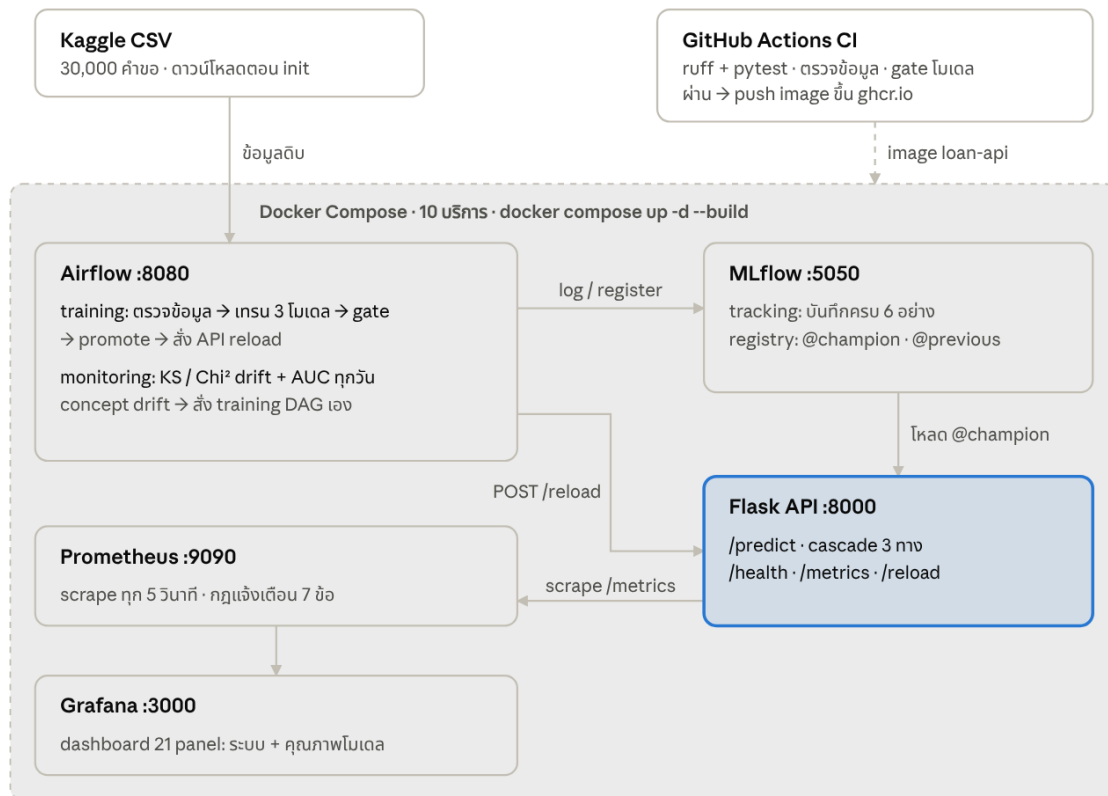
คำถาม	คำตอบ
ถ้าโมเดลทำนายผิด ใครเดือดร้อน	อนุมัติผิด: ธนาคารเสี่ยงหนี้เสีย ปฏิเสธผิด: ลูกค้าเสียโอกาสและธนาคารเสียรายได้ ผลกระทบด้วยต้นทุน 3 : 1 และโซนส่งเจ้าหน้าที่
ตัวชี้วัดธุรกิจเชื่อมกับโมเดลอย่างไร	AUC สูงขึ้น $\rightarrow$ คำขอที่มั่นใจมากขึ้น $\rightarrow$ สัดส่วนที่ตัดสินใจอัตโนมัติสูงขึ้น ส่วน recall ของคลาสปฏิเสธใช้คุมความเสี่ยงหนี้เสีย
ข้อมูลเปลี่ยนได้อย่างไร	Data drift: เศรษฐกิจถดถอย ผู้สมัครรายได้และคะแนนเครดิตต่ำมากขึ้น Concept drift: ธนาคารเปลี่ยนนโยบาย เช่น เลิกปล่อยกู้ระยะเกิน 10 ปี
ต้องตอบเร็วแค่ไหน รองรับเท่าใด	$p95 \leq 200$ ms ที่ 20 คำขอ/วินาที (ช่วงพีคของธนาคารขนาดกลาง) ความจุสูงสุดที่วัดได้ประมาณ 75 คำขอ/วินาที
ถ้าเวอร์ชันใหม่แยกว่า รู้ได้อย่างไร และย้อนกลับอย่างไร	Evaluator เทียบกับ champion บน test ชุดเดียวกัน ถ้าแย่กว่าจะไม่ promote ถ้าพบปัญหาหลัง deploy ใช้ rollback.py หรือ rollback DAG ย้าย alias กลับไป @previous แล้ว API โหลดใหม่ทันที

3. สถาปัตยกรรมระบบ

3.1 แผนภาพสถาปัตยกรรม

ทุกบริการยกขึ้นด้วยคำสั่งเดียว และส่วนต่าง ๆ ส่งโมเดลต่อกันผ่าน MLflow registry เท่านั้น ไม่มีการคัดลอกไฟล์โมเดลด้วยมือ

ทุกส่วนรันใน Docker Compose และส่งต่อกันผ่าน MLflow registry



รูปที่ 1 สถาปัตยกรรมระบบ ทุกส่วนรันใน Docker Compose และส่งต่อโมเดลกันผ่าน MLflow registry

Airflow เทรนและลงทะเบียนโมเดลใน MLflow แล้วสั่ง API โหลด @champion ใหม่ Prometheus ดึง metric จาก API ทุก 5 วินาที รวมถึงผล monitoring DAG ล่าสุด แล้ว Grafana นำไปแสดงผล ส่วน CI บน GitHub ตรวจสอบทุก Pull Request ก่อน merge

เส้นประจาก GitHub Actions หมายถึง image loan-api ที่ผ่าน CI แล้วถูก push ขึ้น ghcr.io ส่วนการรันจากเครื่องเปล่าใช้ docker compose up -d --build ซึ่ง build image ในเครื่อง

ส่วนที่ไม่ได้วาดไว้ในรูป: ขั้นตอนตรวจสอบข้อมูล (example_validator) อยู่ใน training DAG, การแจ้งเตือนเขียนลง alerts.jsonl และส่ง webhook (Discord) ได้, รายงาน Evidently สร้างใน monitoring DAG และ log ของ API เขียนลง logs/api/predictions.jsonl

บริการทั้ง 10 ตัวใน Docker Compose: ได้แก่ postgres, airflow-init, apiserver, scheduler, dag-processor, triggerer, mlflow, api: Flask API, prometheus และ grafana

3.2 เหตุผลการเลือกเครื่องมือ

ตารางที่ 4 เครื่องมือที่ใช้ในแต่ละหน้าที่และเหตุผล

หน้าที่	เครื่องมือ	เหตุผล
Version control	Git + GitHub	มี Pull Request และ Actions ในทีเดียว (Lab 9)
Containerization	Docker Compose	ยก 10 บริการด้วยคำสั่งเดียวบนเครื่องเปล่า (Lab 8, 11)
Data validation	schema.json + src/validation.py	ข้อมูลเป็นตาราง ใช้โค้ดเดียวกันได้ทั้งใน DAG และใน API โดยไม่ต้องติดตั้ง TensorFlow (Lab 11)
Experiment tracking + Model registry	MLflow server	บันทึกครบ 6 อย่าง และใช้ alias ทำ rollback ได้ด้วยการย้ายป้าย (Lab 9)
Pipeline orchestration	Airflow 3	รองรับ branch, dynamic task mapping และ TriggerDagRun สำหรับปิดวงจร retrain (Lab 11)
Model serving	Flask	ต้องควบคุมการตรวจ schema และ cascade เอง ทีมคุ้นเคยจาก Lab 3, 7 และ TF Serving เสริมโมเดล sklearn ไม่ได้
Monitoring	KS / Chi ² + Evidently + Prometheus + Grafana	แยก drift สองแบบ และเฝ้าสถานะระบบแบบ real-time (Lab 10, 11)
CI/CD	GitHub Actions	รันทุก PR ฟรี และ push image ขึ้น ghcr.io ได้ทันที (Lab 9)

3.3 เวอร์ชันไลบรารีหลัก

ไลบรารีทุกตัวล็อกเวอร์ชันไว้ใน uv.lock และ requirements-ml.txt และใช้เวอร์ชันเดียวกันทุก container

ตารางที่ 5 เวอร์ชันของไลบรารีหลัก

ไลบรารี	เวอร์ชัน
Python	>=3.12,<3.13
scikit-learn	>=1.6
pandas	>=2.2
MLflow	>=3.0
Apache Airflow	apache/airflow:3.3.1
Flask	>=3.1
Evidently	==0.7.23

4. ข้อมูลและการตรวจสอบคุณภาพ

4.1 ภาพรวมชุดข้อมูล

ข้อมูลสะอาด ไม่มีค่าว่างและไม่มีแถวซ้ำ (อนุมัติ 57.6% / ปฏิเสธ 42.4%) แต่มี 4 คอลัมน์ที่รบกวนหรือเกิดขึ้นหลังการพิจารณา ซึ่งเราตัดออกก่อนเทรน จากทั้งหมด 36 คอลัมน์ เหลือข้อมูลเข้า 27 คอลัมน์ และสร้างฟีเจอร์เพิ่มอีก 2 ตัว รวมเป็น 29 ฟีเจอร์

4.2 คอลัมน์ที่ไม่ใช้เป็นฟีเจอร์

ตารางที่ 6 คอลัมน์ที่ไม่ใช้เป็นฟีเจอร์ (9 คอลัมน์)

คอลัมน์	เหตุผล / หลักฐาน	การตัดสินใจ
Loan_Approved	เป็นคำตอบ (label)	ใช้เป็นเป้าหมายของโมเดล
Application_ID	รหัสคำขอ ไม่มีความหมายเชิงทำนาย	ใช้ตรวจแถวซ้ำ
Approved_Amount	ผู้ถูกปฏิเสธมีค่า 0 ทุกแถว AUC คอลัมน์เดียว = 1.000	ตัด (รู้หลังอนุมัติ)
Default_Risk	กลุ่ม High ถูกปฏิเสธ 95.1%	ตัด (ธนาคารประเมินภายหลัง)
Interest_Rate	ดอกเบี้ยกำหนดหลังพิจารณา (AUC 0.70)	ตัด
Risk_Score	AUC คอลัมน์เดียว 0.925 มาจากระบบอื่นที่ไม่รู้สูตร	ตัด จากผล ablation (หัวข้อ 5.1)
Gender, Marital_Status	ข้อมูลอ่อนไหว	ไม่ใช้เป็นฟีเจอร์ เก็บไว้ตรวจ fairness
Monthly_Income	เท่ากับ Annual_Income / 12 ทุกแถว	ใช้เป็นกฎตรวจสอบความสอดคล้อง

4.3 การแบ่งข้อมูล

ข้อมูลไม่มีคอลัมน์เวลา จึงแบ่งแบบ stratified ด้วย seed 42 และบันทึก md5 ของไฟล์เป็นเวอร์ชันข้อมูล (บังคับ line ending เป็น LF เพื่อให้ md5 เท่ากันทุกเครื่อง)

- 80% เป็นข้อมูลตัด (24,000 แถว) แบ่งเป็น train 70% / validation 15% / test 15% = 16,800 / 3,600 / 3,600 แถว
- 20% เป็น pool (6,000 แถว) ใช้จำลองคำขอ production 4 สัปดาห์ สัปดาห์ละ 1,500 แถว
- ข้อมูลเสีย 2,000 แถวสำหรับสถิติการตรวจจับ สร้างจาก ข้อมูลเสีย 2,000 แถวได้จากการสุ่มข้อมูลจริง แล้วปรับค่าให้ผิดปกติ เช่น ค่ามากเกินไป ค่าติดลบ และค่า Missing เพื่อใช้ทดสอบระบบจัดการข้อมูลเสีย

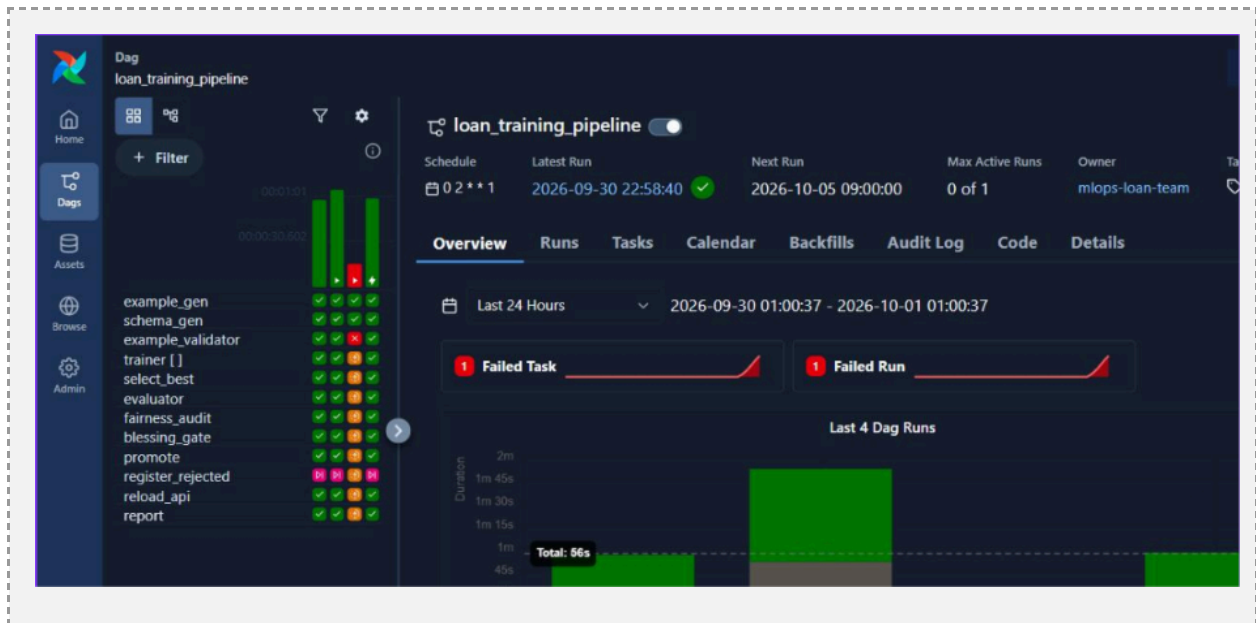
4.4 Schema และการตรวจสอบความผิดปกติ

Schema สร้างจาก train ครั้งเดียวแล้วเก็บเป็น schema.json (แนบเดียวกับ Lab 11) มีกฎ 2 แบบ

- เรียนจากข้อมูล:** ช่วง min-max เมื่อ 20%, หมวดที่เคยเห็น และคอลัมน์ที่บังคับ (เฉพาะที่ลูกค้านำกรอกตอนยื่น)
- ความรู้โดเมน (ไม่เผื่อ):** Credit_Score 300–850, อายุ ≥ 18 , จำนวนเงินห้ามติดลบ, ค่าว่างต่อคอลัมน์ $\leq 5\%$, Application_ID ห้ามซ้ำ และ $\text{Monthly_Income} = \text{Annual_Income} / 12$

สถิติข้อมูลเสีย: ไฟล์ bad_data.csv มีปัญหา 6 อย่าง ได้แก่ Credit_Score 920, รายได้ติดลบ, หมวด "Freelancer" ที่ไม่เคยเห็น, ค่าว่าง 15%, คอลัมน์ Collateral หาย และ Monthly_Income $\neq$ Annual / 12 ระบบจับได้ครบ task example_validator ล้มด้วย AirflowFailException จึงไม่มีการเทรนหรือ

deploy และ on_failure_callback เขียน ALERT ลง alerts.jsonl (ส่ง webhook เช่น Discord ได้เมื่อตั้ง ALERT_WEBHOOK_URL) ฝั่ง API คำขอที่ผิด schema ได้ HTTP 400 พร้อมรายละเอียด



รูปที่ 2 pipeline หยุดและแจ้งเตือนเมื่อป้อน bad_data.csv

4.5 การแปลงข้อมูลและการกัน Training-Serving Skew

การแปลงทั้งหมดอยู่ใน sklearn Pipeline ก่อนเดียวกับโมเดล เขียนไว้ในไฟล์ src/features.py ไฟล์เดียว ซึ่งทั้ง DAG และ API import ไปใช้

1. **LoanFeatureEngineer:** สร้าง Loan_To_Income และ Liability_To_Asset แล้ว clip outlier ที่ quantile 1% / 99% ที่เรียนจาก train (ลบแถวทิ้งไม่ได้ เพราะต้องตอบทุกคำขอ)
2. **ตัวเลข:** SimpleImputer(median) → StandardScaler
3. **หมวดหมู่:** SimpleImputer(most_frequent) → OneHotEncoder(handle_unknown="ignore")

หมวดหมู่ที่ไม่เคยเห็นจะถูกด้าน schema ปฏิเสธก่อน (HTTP 400) ส่วน handle_unknown="ignore" เป็นชั้นป้องกันสำรองไม่ให้โมเดล error ถ้ามีค่าหลุดผ่านมา

โมเดลถูกบันทึกพร้อมโค้ด src/ (MLflow code_paths) และ schema ที่ API ดาวน์โหลดจาก run เดียวกับที่เทรน เมื่อเทียบผลทำนายของโมเดลตอนเทรนกับโมเดลที่โหลดจาก registry ความต่างสูงสุดเท่ากับ 0 ทุกแถว และ ROC-AUC ที่วัดผ่าน API บนคำขอใหม่ 500 รายการได้ 0.900 ใกล้เคียงกับชุดทดสอบ (0.899)

5. การพัฒนาโมเดล

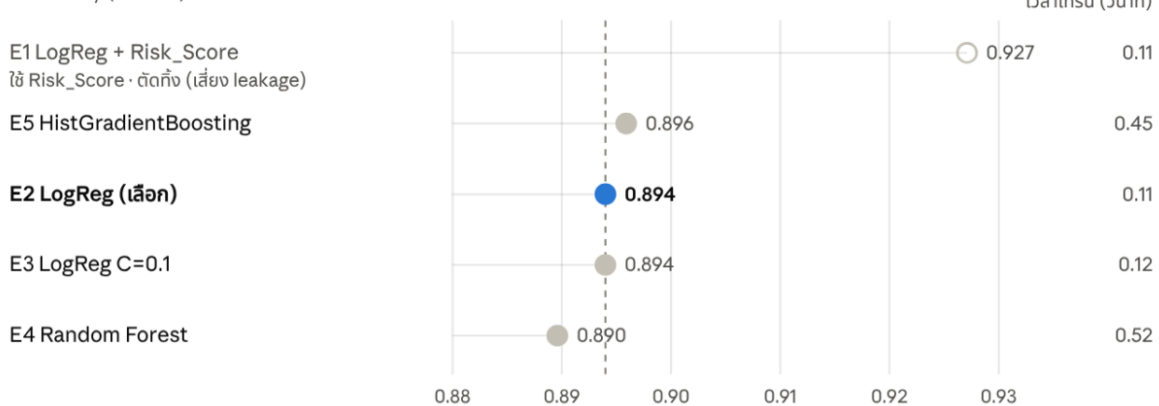
5.1 การทดลองและการเลือกโมเดล

เราเลือก Logistic Regression (E2) เพราะแม่นยำใกล้เคียงโมเดลที่ซับซ้อนกว่า แต่เทรนและทำนายเร็วกว่า ไฟล์เล็กกว่า และอธิบายเหตุผลผลการปฏิเสธได้ บนชุดทดสอบได้ ROC-AUC 0.899

Logistic Regression แบบใกล้เคียงโมเดลที่ซับซ้อนกว่า และเทรนเร็วกว่า 4 เท่า

ROC-AUC บน validation (4,500 คำขอ) · เส้นประ = โมเดลที่เลือก

E0 Dummy (พื้นชั้นต่ำ) = 0.500



รูปที่ 3 ROC-AUC ของแต่ละการทดลองบน validation ของ notebook ต้นแบบ (4,500 คำขอ = 15% ของข้อมูลทั้งหมด) ส่วน production pipeline ใช้ validation 3,600 คำขอตามหัวข้อ 4.3

- E0 เทียบ E2: baseline ที่หายคลาสที่เยอะที่สุดได้ AUC 0.500 แสดงว่าโมเดลเรียนรู้ได้จริง
- Risk_Score (E1 เทียบ E2): Risk_Score ทำให้ AUC สูงขึ้น 0.033 แต่เป็นคะแนนจากระบบอื่นที่ไม่รู้สูตร ถ้าสูตรเปลี่ยน โมเดลจะพังตามโดยไม่รู้ตัว จึงตัดทิ้ง
- Regularization (E2 เทียบ E3): ปรับ C เป็น 0.1 ได้ AUC เท่าเดิม (0.894) จึงไม่เปลี่ยน
- การเลือกโมเดล: HistGradientBoosting (E5) ดีกว่าเพียง 0.002 ส่วน Random Forest (E4) แยกว่าและไฟล์ใหญ่ 50.6 MB ใน pipeline จริง select_best ตัดตัวที่เกินงบ latency ก่อน แล้วเลือกตาม AUC ถ้าต่างกันไม่ถึง 0.003 จะเลือกตัวที่ง่ายกว่า

5.2 Threshold ตามต้นทุน

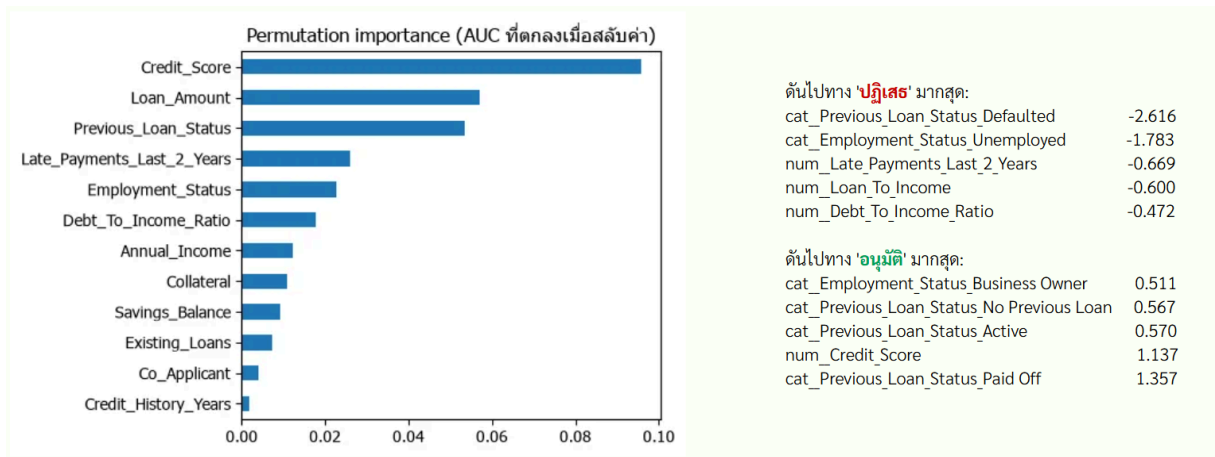
อนุมัติคิดคนแพงกว่าปฏิเสธคิดคน 3 เท่า threshold ที่ต้นทุนต่ำสุดบน validation คือ 0.77 ใน notebook และ 0.72 ใน pipeline ทำให้ recall ของคลาส ปฏิเสธขึ้นจาก 0.73 เป็น 0.89–0.92 (CI วัดได้ 0.8885)

5.3 การอธิบายผล

ตารางที่ 7 Coefficient ของ Logistic Regression ที่มีผลมากที่สุด

ต้นทุนทางปฏิเสธ	coef	ต้นทุนทางอนุมัติ	coef
เคยผิดนัด (Defaulted)	-2.62	ประวัติชำระครบ (Paid Off)	+1.36
ว่างงาน (Unemployed)	-1.78	Credit_Score	+1.14
จ่ายช้าใน 2 ปี	-0.67	มีสินเชื่อยู่ (Active)	+0.57
Loan_To_Income	-0.60	ไม่เคยกู้ (No Previous Loan)	+0.57

ค่า coefficient ของฟีเจอร์ตัวเลขจำนวนหลัง StandardScaler จึงอ่านเป็นผลต่อ 1 ส่วนเบี่ยงเบนมาตรฐาน ส่วนฟีเจอร์หมวดหมู่เป็นผลเมื่อเทียบกับการไม่อยู่ในหมวดนั้น ค่าลบต้นทุนทางปฏิเสธ ค่าบวกต้นทุนทางอนุมัติ



รูปที่ 3 Permutation importance

5.4 Fairness

โมเดลไม่ได้ใช้เพศเป็นฟีเจอร์ อัตราอนุมัติที่โมเดลทำนายต่างกันระหว่างเพศ 0.017 (เกณฑ์ ≤ 0.10) และ ROC-AUC ของเพศชาย 0.8967 / เพศหญิง 0.8968
 ตอนนี้ตรวจเฉพาะเพศ ยังไม่ได้ตรวจ Marital_Status (ดูข้อจำกัด)

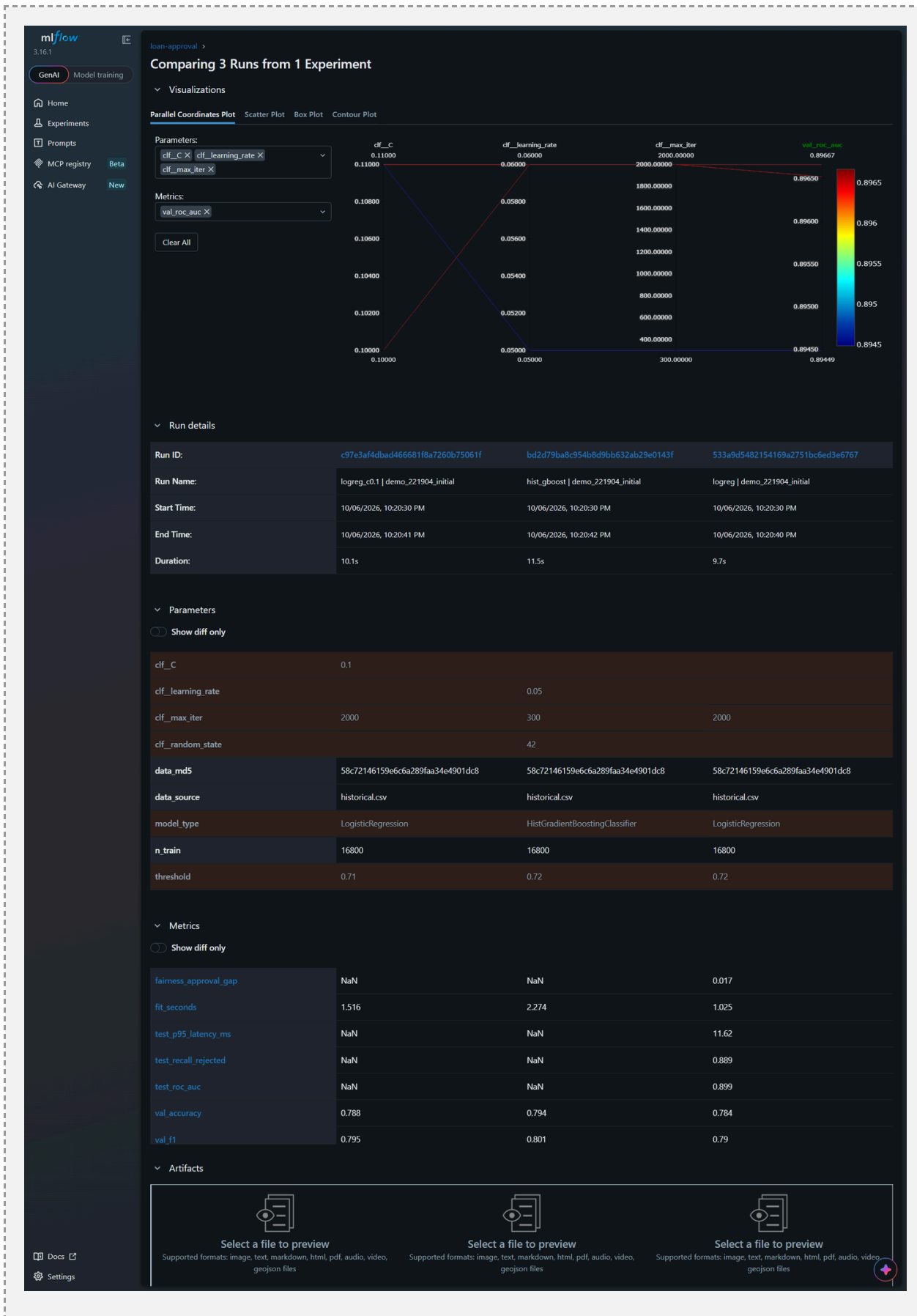
6. Experiment Tracking และ Model Registry

6.1 สิ่งที่บันทึกในทุกการทดลอง

ทุกการเทรน (ทั้งใน notebook, DAG และ CI) บันทึกครบ 6 อย่างลง MLflow server และทุกโมเดลที่ผ่านการประเมินถูกลงทะเบียนพร้อมสถานะ

ตารางที่ 8 สิ่งที่บันทึกลง MLflow

สิ่งที่บันทึก	วิธี
เวอร์ชันโค้ด	tag git_commit (อ่านจาก .git โดยตรง เพราะใน container ไม่มีคำสั่ง git) และ airflow_run_id
เวอร์ชันข้อมูล	param data_md5, data_source, n_train
ไฮเปอร์พารามิเตอร์	param ของโมเดลแต่ละตัว เช่น C ของ Logistic Regression
ตัวชี้วัด	validation AUC / F1 / accuracy / recall คลาสปฏิเสธ, p95 latency, test AUC และ fairness gap
ไฟล์ผลลัพธ์	โมเดล (Pipeline ทั้งก่อน + src/), schema.json และ confusion matrix
สภาพแวดล้อม	environment.json (Python, OS, เวอร์ชันไลบรารี) + uv.lock

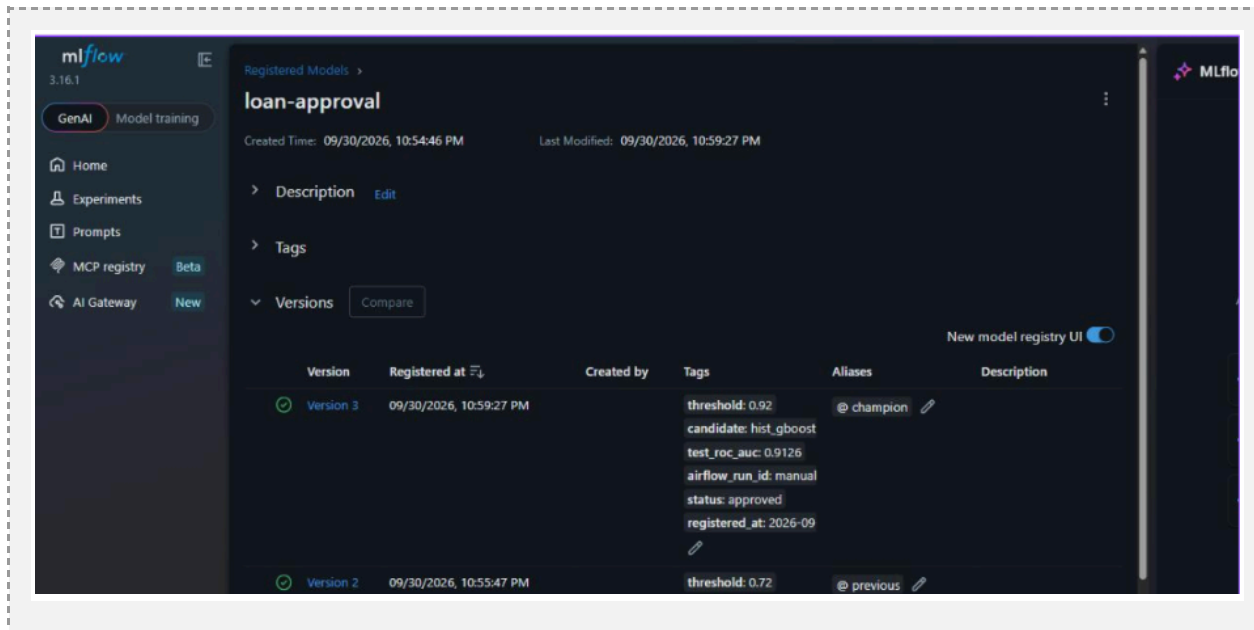


6.2 ทะเบียนโมเดลและด้านตรวจก่อนอนุมัติ

ทะเบียนโมเดลใช้ MLflow alias แบบ Lab 9 และใช้ tag บอกลักษณะ

- **Alias:** @champion คือตัวที่ให้บริการ, @previous คือ champion ก่อนหน้า (ไว้ rollback) และ @challenger คือตัวที่ไม่ผ่านด่าน
- **Tag:** status = approved / rejected / rolled_back พร้อม threshold, test_roc_auc และ airflow_run_id

ด้านตรวจก่อนอนุมัติ: task blessing_gate เลือกเส้นทาง promote หรือ register_rejected ตามผล gating metric ทุกข้อ



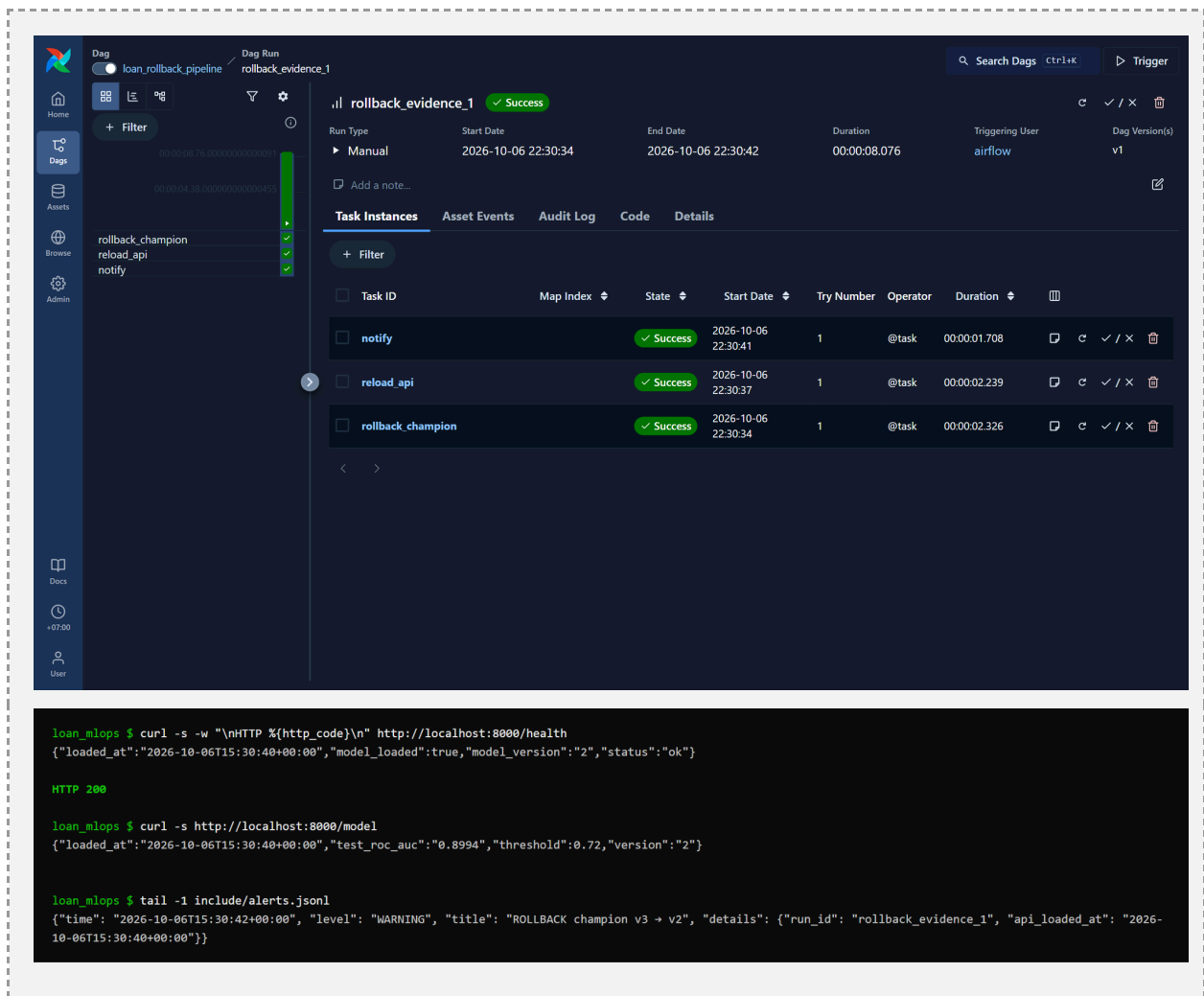
รูปที่ 5 ทะเบียนโมเดลใน MLflow

6.3 Rollback

ด้วยสคริปต์: scripts/rollback.py ย้าย @champion กลับไป @previous ติด tag rolled_back แล้วเรียก /reload ได้สัริตแล้วจาก v3 กลับ v2 API โหลดเวอร์ชันเก่าทันทีโดยไม่ต้อง build หรือ restart container

ผ่านหน้าเว็บ Airflow: กดย้อนได้จาก DAG loan_rollback_pipeline (rollback_champion → reload_api → notify) โดยไม่ต้องเปิด terminal ถ้าเว้นช่อง to_version วางจะกลับไป @previous หรือจะระบุเวอร์ชันที่ต้องการก็ได้

- DAG นี้สั่งเองเท่านั้น ไม่มีรอบอัตโนมัติ และตั้ง max_active_runs = 1 กันการกดซ้อนจน alias สลับไปมา
- task rollback ไม่ retry เพราะการลงซ้ำจะสลับกลับไปเวอร์ชันเดิม
- task reload_api ตรวจสอบว่า API โหลดเวอร์ชันที่คาดหวังจริง ถ้าไม่ตรงจะล้ม ส่วน notify ส่ง ALERT ระดับ WARNING ทุกครั้ง
- ต่างจากสคริปต์ตรงที่ Airflow เก็บบันทึกว่ากดเมื่อไรและผลเป็นอย่างไร



Flask API ใน Docker ผ่าน SLO: p95 = 66.7 ms ที่ 20 คำขอ/วินาที (เกณฑ์ 200 ms) และ 5xx = 0%

ตารางที่ 9 Endpoint ของ API

15

```

loan_mlops $ curl -s -w "\nHTTP %{http_code}\n" http://localhost:8000/health
{"loaded_at":"2026-10-06T15:25:16+00:00","model_loaded":true,"model_version":"3","status":"ok"}

HTTP 200

loan_mlops $ curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @req_auto_approve.json
{"approved":true,"decision":"AUTO_APPROVE","model_version":"3","probability_approve":0.9382}

HTTP 200

loan_mlops $ curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @req_manual_review.json
{"approved":false,"decision":"MANUAL_REVIEW","model_version":"3","probability_approve":0.4619}

HTTP 200

loan_mlops $ curl -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @req_auto_reject.json
{"approved":false,"decision":"AUTO_REJECT","model_version":"3","probability_approve":0.4234}

HTTP 200

# ข้อมูลเดิม: Credit_Score = 999 และ Employment_Status = "Freelancer"
loan_mlops $ curl -s -o /dev/null -w "HTTP %{http_code}\n" -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @bad_request.json
HTTP 400
loan_mlops $ curl -s -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d @bad_request.json | python -c "import sys,json;
print(json.dumps(json.load(sys.stdin), ensure_ascii=False))"
{"details": [{"Credit_Score: ผิดกฎโดยนัย ต้องอยู่ใน [300, 850] แต่พบ 999 ถึง 999", "Employment_Status: พบหมวดที่ไม่เคยเห็น ['Freelancer']"}, {"error": "ข้อมูลไม่ผ่าน schema"}]

```

รูปที่ 7 การเรียก API จากภายนอก container

7.2 รูปแบบการให้บริการ: real-time + cascade

ลูก้ารอผลอยู่จึงต้องตอบทันที (Lab 7) คำขอที่ผิด schema ได้ 400 โดยไม่ทำนาย ส่วนคำขอที่ผ่านจะตัดสินใจตามความน่าจะเป็นที่จะได้รับอนุมัติ

- $\geq \text{threshold} + 0.10 \rightarrow \text{AUTO_APPROVE}$
- $< \text{threshold} - 0.10 \rightarrow \text{AUTO_REJECT}$
- ระหว่างนั้น $\rightarrow \text{MANUAL_REVIEW}$ ส่งเจ้าหน้าที่ (human-in-the-loop)

ที่ threshold 0.72 โชน AUTO_REJECT คือความน่าจะเป็นต่ำกว่า 0.62 ซึ่งรวมช่วง 0.50–0.62 ที่ไม่เคลือบเงาไปทางอนุมัติด้วย เป็นผลจากการให้ต้นทุนอนุมัติ ผิดสูงกว่า 3 เท่า งานถัดไปคือเลือกความกว้างของโชนจากต้นทุนบน validation แทนค่าคงที่ ± 0.10 นอกจากนี้ยังมี /predict/batch สำหรับคำขอค้างที่ไม่ต้องตอบทันที

7.3 SLO

ตารางที่ 10 SLO ของระบบ

SLI	เป้าหมาย	วัดด้วย	ผลล่าสุด
p95 latency ของ /predict	≤ 200 ms ที่ 20 คำขอ/วินาที	scripts/load_test.py และ alert LoanLatencyP95AboveSLO (ช่วงเวลา 6 ต.ค.)	66.7 ms (ผ่าน)
อัตรา 5xx	$\leq 1\%$	alert LoanErrorRateAboveSLO	0% (ผ่าน)
ความพร้อมให้บริการ	/health ตอบ 200	alert LoanApiDown (ไม่ตอบ 30 วินาที) และ Docker healthcheck ทุก 15 วินาที	54.6 ms (ผ่าน)

7.4 ผลวัดจริง

วัดด้วย scripts/load_test.py โดยใช้คำขอจริงจาก pool ปนคำขอเสีย 5%

ตารางที่ 11 Latency และ throughput ที่วัดได้

โหมด	p50	p95	Throughput	5xx
คงที่ 20 คำขอ/วินาที นาน 30 วินาที	23.7 ms	66.7 ms	20.0 คำขอ/วินาที	0%
Stress 10 ผู้ใช้พร้อมกัน (ความสูงสุด)	131.7 ms	186.3 ms	75.1 คำขอ/วินาที	0%
Stress 40 ผู้ใช้พร้อมกัน (สาธิต alert)	1,133 ms	1,328 ms	35.0 คำขอ/วินาที	0%

- ทูกรอบวัดขณะ API เสรีโฟมเดล เวอร์ชัน 2 ชนิด LogisticRegression (threshold 0.72)
- ค่าขอเสีย 5% ตอบ 400 เร็วกว่าค่าขอปกติ ตัวเลขในตารางจึงอาจต่ำกว่าค่าของค่าขอปกติล้วน
- เมื่อวัดซ้ำโดยไม่ปนค่าขอเสีย ได้ p50 22.0 ms และ p95 54.9 ms ต่างจากรอบที่ปนค่าขอเสีย (20.7 / 54.6 ms) ไม่เกิน 1.3 ms ค่าขอเสีย 5% จึงแทบไม่มีผลต่อตัวเลข
- ที่ 40 ผู้ใช้ throughput ลดเหลือ 35 คำขอ/วินาที เพราะ ที่ 40 ผู้ใช้ throughput อยู่ที่ 36 คำขอ/วินาที ใกล้เคียงกับที่ 10 ผู้ใช้ (39 คำขอ/วินาที) แสดงว่าระบบถึงเพดานความจุแล้ว ผู้ใช้ที่เพิ่มขึ้นจึงไม่ได้งานเพิ่ม แต่ต้องรอคิวมากขึ้น p50 จึงเพิ่มจาก 261 ms เป็น 1,133 ms สาเหตุที่น่าจะเป็นคือ API เป็น Flask โปรเซสเดียว และ Python ประมวลผลได้ทีละ thread (GIL) เมื่อแต่ละคำขอใช้เวลาประมวลผลราว 26 ms ความสูงสุดจึงอยู่ที่ประมาณ 38 คำขอ/วินาที
- ค่า p95 ที่ Prometheus แจ้งเตือน (1.9 วินาที) ต่างจาก load_test (1,328 ms) เพราะ Prometheus ประมาณจาก histogram bucket ฝั่งเซิร์ฟเวอร์ และคำนวณในช่วงเวลาที่ต่างกัน

7.5 Logs

ทุกคำขอ /predict เขียน JSON หนึ่งบรรทัด (เวลา, status, latency, request, response และเวอร์ชันโมเดล) ลง stdout และ logs/api/predictions.jsonl log นี้เก็บ request ทั้งหมดซึ่งมีข้อมูลส่วนบุคคล ในงานจริงต้อง mask ก่อนเก็บ (ดูข้อจำกัด)

8. Monitoring และนโยบาย Retrain

ระบบแยก data drift กับ concept drift ได้จริง ใน W2 พีเจอรเปลี่ยนไป 63% แต่ AUC ยังดี จึงแจ้งเตือนอย่างเดียว ส่วน W3 ข้อมูลเข้าไม่เปลี่ยนเลย แต่ AUC ตกประมาณ 0.10 (0.899 → 0.801) จึง retrain อัตโนมัติ monitoring DAG รันทุกวัน

8.1 การแยก Data Drift กับ Concept Drift

ตารางที่ 12 วิธีตรวจและจำลอง drift สองแบบ

หัวข้อ	Data drift	Concept drift
อะไรเปลี่ยน	ข้อมูลเข้า $P(X)$	ความสัมพันธ์ $P(y X)$
ตรวจด้วย	KS test (ตัวเลข) และ Chi-square (หมวดหมู่) ไม่ต้องรอเฉลย	AUC บนเฉลยจริง เทียบกับ AUC ตอน deploy
เกณฑ์แจ้งเตือน	$p < 0.01$ ในเกิน 20% ของฟีเจอร์	AUC ตกเกิน 0.05
จำลองด้วย	เศรษฐกิจถดถอย: สุ่มแบบถ่วงน้ำหนักให้ได้คะแนนเครดิตและรายได้ต่ำมากขึ้น ($P(y X)$ เท่าเดิม)	ธนาคารเลิกปล่อยกู้ระยะเกิน 120 เดือน (เปลี่ยน label โดยไม่แตะข้อมูลเข้า)

8.2 ผลสาธิต

รันจากศูนย์ด้วย scripts/demo.py

ตารางที่ 13 ผลการสาธิต monitoring 4 สัปดาห์

Batch	โมเดล	Drift share (KS / Chi ²)	AUC (ตอน deploy)	ผลลัพธ์
W1 ปกติ	v2 logreg	0%	0.895 (0.899)	ไม่มี alert
W2 data drift	v2 logreg	63%	0.918 (0.899)	ALERT data drift ไม่ retrain
W3 concept drift	v2 logreg	0%	0.801 (0.899)	ALERT concept drift → retrain → v3
W4 หลัง retrain	v3 hist_gboost	0%	0.931 (0.913)	ไม่มี alert

8.3 นโยบาย Retrain

ตารางที่ 14 นโยบาย retrain

เงื่อนไข	การกระทำ
AUC ตกเกิน 0.05 เมื่อเฉลยมาถึง	Monitoring DAG สั่ง training DAG ด้วย TriggerDagRunOperator เทรนด้วยข้อมูลล่าสุดที่มีเฉลย (sliding window) เพราะเฉลยเก่าไม่ตรงกับนโยบายแล้ว
Data drift เกิน 20% แต่ AUC ปกติ	แจ้งเตือนทีมให้ตรวจ ยังไม่ retrain
ข้อมูลเสีย	หยุด ห้าม retrain และแจ้งเตือน
ตามรอบ	training DAG ทุกวันจันทร์ 02:00 UTC (09:00 น. เวลาไทย)

โมเดลใหม่ทุกตัวต้องผ่าน gate เดิมและชนะ champion บน test ชุดเดียวกันก่อน promote

8.4 รายงาน Evidently

นอกจาก KS / Chi² ที่ให้ตัวเลขสำหรับตัดสินใจอัตโนมัติ monitoring DAG มี task drift_report สร้างรายงาน data drift แบบ HTML ด้วย Evidently (DataDriftPreset) ให้เห็นว่า drift เกิดที่ฟีเจอร์ไหนและการกระจายเปลี่ยนอย่างไร รายงานใช้ 27 คอลัมน์ที่โมเดลใช้จริง และสุ่ม reference 5,000 แถวจากข้อมูลอดีตเพื่อให้สร้างได้เร็ว task จะ log จำนวนและสัดส่วนคอลัมน์ที่ Evidently นับว่า drift เทียบกับสัดส่วนจาก KS / Chi² ของเรา

รายงานนี้เป็นส่วนเสริม ถ้าสร้างไม่สำเร็จจะไม่ทำให้ DAG ล้ม และตัวเลขที่ใช้ตัดสิน alert หรือ retrain ยังมาจาก check_batch มีเทสต์ 2 ข้อ คือ ข้อมูลที่การกระจายเดิมต้อง drift น้อยกว่า 20% และข้อมูลที่เลื่อนค่า (เศรษฐกิจถดถอยจำลอง) ต้องถูกจับได้อย่างน้อย 3 คอลัมน์



รูปที่ 8 รายงาน data drift จาก Evidently

8.5 Prometheus และ Grafana

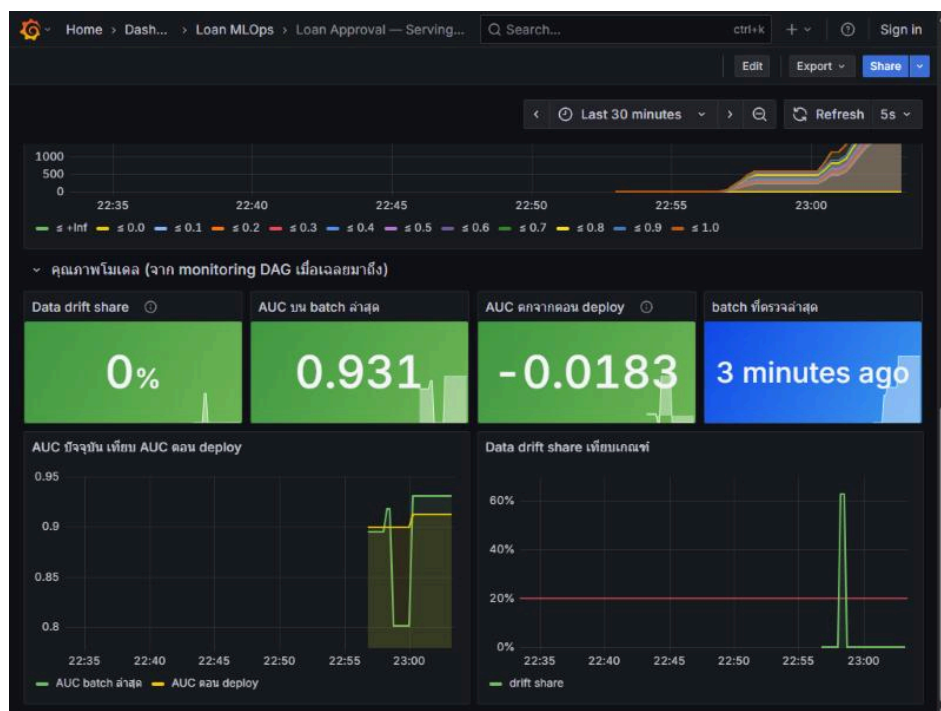
กฎแจ้งเตือนมี 7 ข้อ เกณฑ์อ่านจาก metric loan_config_threshold ที่ API ส่งออกจาก src/config.py

ตารางที่ 15 กฎแจ้งเตือนใน Prometheus

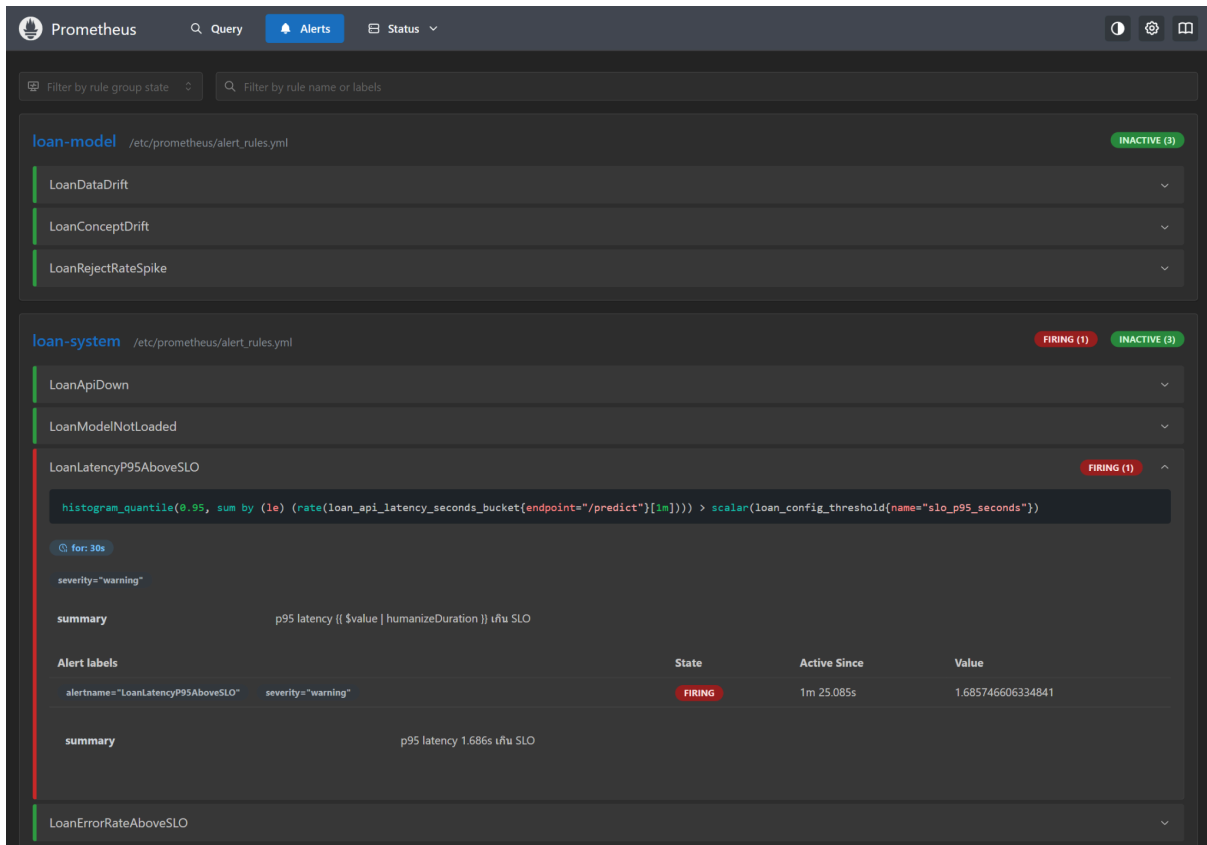
กฎ	ด้าน	เงื่อนไข
LoanApiDown	ระบบ	API ไม่ตอบ 30 วินาที
LoanModelNotLoaded	ระบบ	ยังไม่มีโมเดลนาน 1 นาที
LoanLatencyP95AboveSLO	ระบบ	p95 > 200 ms (ทดสอบให้ firing แล้วที่ 1.9 วินาที)

กฎ	ด้าน	เงื่อนไข
LoanErrorRateAboveSLO	ระบบ	5xx > 1%
LoanDataDrift	โมเดล	drift share > 20%
LoanConceptDrift	โมเดล	AUC ตก > 0.05
LoanRejectRateSpike	โมเดล	AUTO_REJECT > 70% ใน 5 นาที (prediction drift ไม่ต้องรอเฉลี่ย)

Dashboard มี 18 แผง แบ่งเป็น 3 กลุ่ม คือ สถานะระบบ การตัดสินใจของโมเดล และคุณภาพโมเดล metric เฉพาะเจาะจงคือสัดส่วนปฏิเสธอัตโนมัติแยกตามสถานะการจ้างงาน ซึ่งกลุ่ม Unemployed ถูกปฏิเสธ 94.9% และ Contract 62.1% สอดคล้องกับ coefficient ของ Unemployed ที่ติดลบมาก (-1.78) แผงนี้ใช้เฝ้าดูว่ามีกลุ่มใดถูกปฏิเสธผิดปกติหลัง retrain



รูปที่ 9 Grafana กลุ่มคุณภาพโมเดลหลัง retrain (W4): AUC บน batch ล่าสุด 0.931 และ drift share 0%



รูปที่ 10 Prometheus alert LoanLatencyP95AboveSLO อยู่ในสถานะ FIRING ระหว่าง stress test 40 ผู้ใช้



รูปที่ 11 สัดส่วนปฏิเสธอัตโนมัติแยกตามสถานะการจ้างงาน

9. CI/CD, Pipeline และการทำซ้ำ

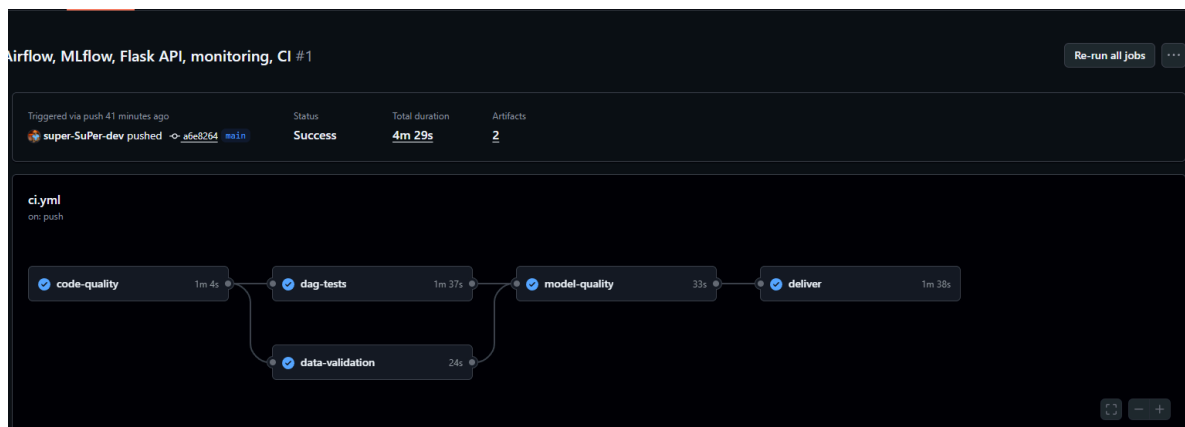
9.1 CI/CD

CI บน GitHub Actions ตรวจสอบ 3 ด้าน (คุณภาพโค้ด ข้อมูล และโมเดล) และส่งมอบ image รอบแรกบน GitHub ผ่านครบ 5 job ใน 4 นาที 29 วินาที

ตารางที่ 16 Job ใน CI

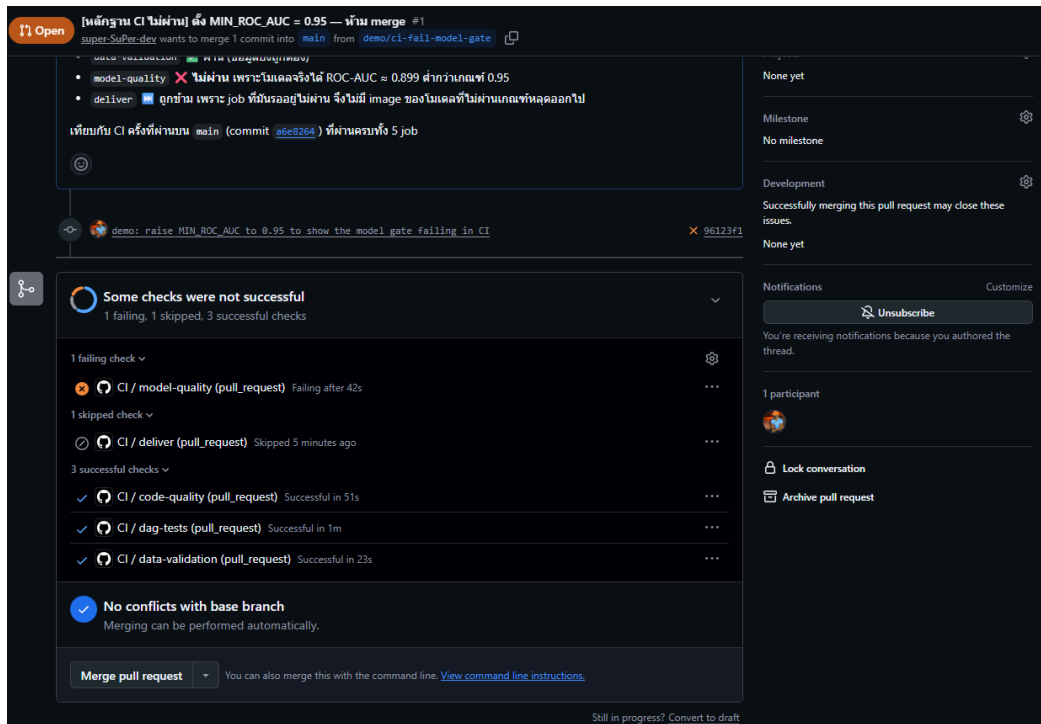
Job	ตรวจสอบอะไร	รอบแรก
code-quality	ruff + pytest (ปัจจุบัน 30 เทสต์: การแปลงข้อมูล, schema, โมเดล, API, monitoring, data_prep, สคริปต์ test case และ Evidently)	ผ่าน 64 วินาที
data-validation	ดาวน์โหลดข้อมูลจริง ตรวจสอบ schema และข้อมูลเสียต้องถูกจับได้อย่างน้อย 5 ปัญหา	ผ่าน 24 วินาที
model-quality	เทรนบนข้อมูลจริง ต้องผ่าน gating ทุกข้อและ fairness	ผ่าน 33 วินาที
dag-tests	build image Airflow แล้วรันเทสต์ DAG (ปัจจุบัน 8 เทสต์)	ผ่าน 97 วินาที
deliver	build image API, smoke test และ push ขึ้น ghcr.io เมื่อ merge เข้า main	ผ่าน 98 วินาที

จำนวนเทสต์ในตารางเป็นของปัจจุบัน ตอน CI รอบแรกยังมีน้อยกว่านี้ เพราะเทสต์ส่วนหนึ่งเพิ่มเข้ามาใน PR #12-#14



รูปที่ 12 CI รอบแรกบน main ผ่านครบ 5 job

หลักฐานครั้งที่ไม่ผ่าน: ใน PR #1 ตั้ง MIN_ROC_AUC = 0.95 ด้าน model-quality คืน exit code 1 (AUC 0.8994 ไม่ผ่าน) และ job deliver ถูกข้าม จึงไม่มี image ของโมเดลที่ไม่ผ่านเกณฑ์หลุดออกไป



รูปที่ 13 PR #1 สาธิต CI ไม่ผ่าน: model-quality ล้มและ deliver ถูกข้าม

model-quality summary	
❌ Model quality gate (ข้อมูลจริง)	
รายการ	ผล
โมเดลที่เลือก	logreg
threshold	0.72
ROC-AUC (≥ 0.95)	0.8994
Recall ปฏิเสธ (≥ 0.8)	0.8885
p95 latency ms (≤ 25.0)	18.26
ขนาดโมเดล MB (≤ 50.0)	0.010
Fairness: ส่วนต่างอัตราอนุมัติ (≤ 0.1)	0.0171
ผ่าน roc_auc	ไม่ผ่าน
ผ่าน recall_rejected	ผ่าน
ผ่าน p95_latency	ผ่าน
ผ่าน model_size	ผ่าน
ผ่าน vs_champion	ผ่าน
Job summary generated at run-time	
Annotations 1 error, 4 warnings, and 4 notices model-quality Process completed with exit code 1.	

รูปที่ 14 Job summary ของ model-quality: ด้าน roc_auc ไม่ผ่าน ส่วนด้านอื่นผ่าน

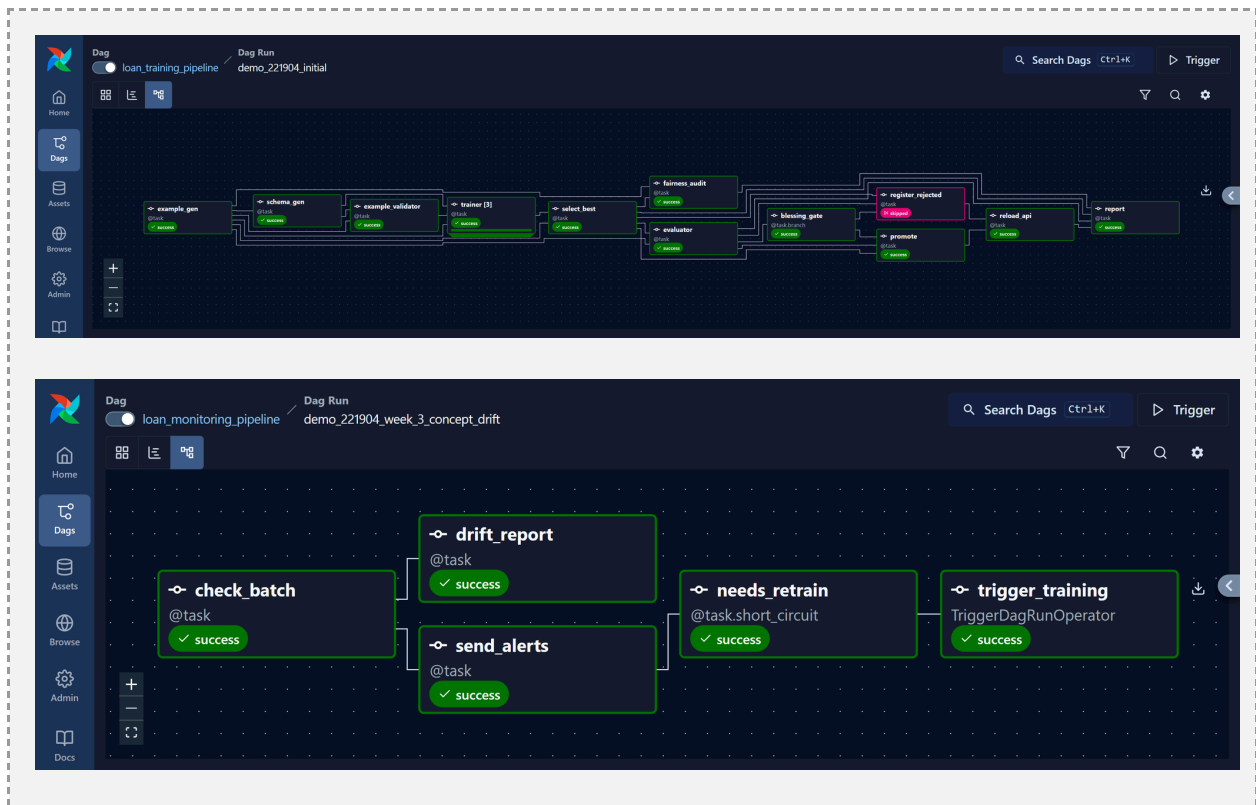
9.2 Pipeline แบบ DAG

ใช้ Airflow 3 ต่อยอดจาก Lab 11 มี 3 DAG โดย 2 DAG หลักสั่งรันซ้ำทั้งกระบวนการได้ และ rollback DAG สั่งเองเท่านั้น

- **loan_training_pipeline** (14 task): example_gen $\rightarrow$ schema_gen $\rightarrow$ example_validator $\rightarrow$ trainer $\times$ 3 (dynamic mapping ขนานกัน) $\rightarrow$ select_best $\rightarrow$ evaluator + fairness_audit $\rightarrow$ blessing_gate $\rightarrow$ promote / register_rejected $\rightarrow$ reload_api $\rightarrow$ report
- **loan_monitoring_pipeline** (รันทุกวัน): check_batch $\rightarrow$ send_alerts $\rightarrow$ needs_retrain (short circuit) $\rightarrow$ trigger_training และอีกสาขาคือ drift_report (รายงาน Evidently)

- `loan_rollback_pipeline` (สั่งเองเท่านั้น): `rollback_champion` → `reload_api` → `notify`

ตรรกะทั้งหมดอยู่ใน `src/` ไม่ผูกกับ Airflow จึงทดสอบด้วย `pytest` ได้ตรง ๆ



รูปที่ 15 DAG ใน Airflow

9.3 การทำซ้ำและการรันจากเครื่องเปล่า

`docker compose up -d --build` ยก 10 บริการและดาวน์โหลดข้อมูลเอง โสภาริทุกตัวล็อกเวอร์ชันใน `uv.lock` และ `requirements-ml.txt` และใช้ seed 42 ทุกจุด เราลบทุกอย่างแล้วรันจากศูนย์ 2 ครั้ง ได้ md5 ข้อมูลและ test AUC (0.8994) เท่าเดิม

ขั้นตอนรัน (รายละเอียดอยู่ใน README ของ repository)

1. `git clone github.com/super-SuPer-dev/loan-mlops && cd loan_mlops`
2. `uv sync`
3. `docker compose up -d --build`
ครั้งแรกใช้เวลาหลายนาที (build image + ดาวน์โหลดข้อมูลจาก Kaggle)
4. เปิด Airflow ที่ `localhost:8080` แล้ว trigger `loan_training_pipeline`
5. เรียก API ที่ `localhost:8000` ดู MLflow ที่ `localhost:5050`, Prometheus ที่ `localhost:9090` และ Grafana ที่ `localhost:3000`

โครงสร้าง repository

- `src/` ตรรกะทั้งหมด (`config.py`, `features.py`, `validation.py`, `monitoring.py`, `drift_report.py`, `steps.py`)

- scripts/ สคริปต์ทำงาน (demo.py, rollback.py, load_test.py, run_test_cases.py)
- tests/ ชุดเทสต์ pytest (รวม test_dags.py)
- docs/ เอกสารประกอบ (test_cases.md) และ reports/ ผลการรัน (test_cases.json)


loan-mlops
Public
Wi

main
5 Branches
0 Tags

Add file
Code

techineethamoontree เพิ่มรายงานฉบับส่งและสไลด์ (PDF) (#15) ✓		c9fe1ef · 2 days ago	9 Commits
📁 .github	เพิ่มเทสต์ monitoring/data_prep และแสดง coverage ใน CI (#13)	4 days ago	
📁 dags	เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG...	3 days ago	
📁 docker	Loan approval MLOps pipeline: Airflow, MLflow, Flask API, mo...	last week	
📁 docs	เพิ่มรายงานฉบับส่งและสไลด์ (PDF) (#15)	2 days ago	
📁 monitoring	Loan approval MLOps pipeline: Airflow, MLflow, Flask API, mo...	last week	
📁 notebooks	Loan approval MLOps pipeline: Airflow, MLflow, Flask API, mo...	last week	
📁 scripts	เพิ่มสคริปต์ reset ระบบ, .env.example และเช็กลิสต์วันอาทิตย์ (#11)	3 days ago	
📁 serving	fix: keep AUTO_APPROVE reachable at high thresholds; stabili...	last week	
📁 src	เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG...	3 days ago	
📁 tests	เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG...	3 days ago	
📄 .dockerignore	Loan approval MLOps pipeline: Airflow, MLflow, Flask API, mo...	last week	
📄 .env.example	เพิ่มสคริปต์ reset ระบบ, .env.example และเช็กลิสต์วันอาทิตย์ (#11)	3 days ago	
📄 .gitattributes	Loan approval MLOps pipeline: Airflow, MLflow, Flask API, mo...	last week	
📄 .gitignore	เพิ่มสคริปต์ reset ระบบ, .env.example และเช็กลิสต์วันอาทิตย์ (#11)	3 days ago	
📄 CONTRIBUTING.md	docs: evidence screenshots, contribution guide and PR templ...	last week	
📄 README.md	Loan approval MLOps pipeline: Airflow, MLflow, Flask API, mo...	last week	
📄 docker-compose.yml	Loan approval MLOps pipeline: Airflow, MLflow, Flask API, mo...	last week	
📄 pyproject.toml	เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG...	3 days ago	
📄 requirements-ml.txt	เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG...	3 days ago	
📄 uv.lock	เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG...	3 days ago	

9.4 Git และการทำงานเป็นทีม

commit แรก (a6e8264) วางโครงทั้งระบบและ push เข้า main โดยตรง หลังจากนั้นทุกงานทำบน branch แยกแล้วเปิด Pull Request เข้า main (PR #2 และ #9-#15) CI รันทุก PR และทุก PR ที่ merge ผ่านครบ 5/5 job

All pull requests

New pull request

is:pr state:closed	🔍
☐ Open 1 ☒ Closed 8	Author ▾ Label ▾ Projects ▾ Milestones ▾ Reviews ▾ Assignee ▾ ☐ Newest
☐ docs: รายงานฉบับส่ง + สไลด์ #15 - by techineethamoonree was merged yesterday - ✓ 5/5	1
☐ feat: รายงาน drift ด้วย Evidently #14 - by jakrapatiam-create was merged 2 days ago - 5 tasks done - ✓ 5/5	1
☐ เพิ่มเทสต์ monitoring/data_prep และแสดง coverage ใน CI #13 - by Tammy-AP was merged 3 days ago - 5 tasks done - ✓ 5/5	1
☐ เพิ่มสคริปต์รัน test case ผ่าน API สำหรับวันนำเสนอ #12 - by naphat67 was merged 3 days ago - ✓ 5/5	1
☐ feat: ความพร้อมวันสาคิล (reset, .env.example, แจ้งเตือน Discord) #11 - by super-SuPer-dev was merged 2 days ago - 5 tasks done - ✓ 5/5	1
☐ feat: Rollback DAG ย้อนเวอร์ชันในเดลด้วยปุ่มเดียว #10 - by super-SuPer-dev was merged 2 days ago - 5 tasks done - ✓ 5/5	
☐ fix: cascade ไม่มี AUTO_APPROVE เมื่อ threshold สูง + เทสต์ latency ไม่มี #9 - by super-SuPer-dev was merged 5 days ago - ✓ 5/5	
☐ docs: หลักฐานภาพหน้าจอ + วิธีทำงานเป็นทีมผ่าน PR #2 - by super-SuPer-dev was merged 5 days ago - ✓ 5/5	

รูปที่ 16 Pull Request ที่ merge แล้ว ผ่าน CI ครบ 5/5 ทุกรายการ

Commits

main ▾	All users ▾ All time ▾
Commits on Oct 4, 2026	เพิ่มรายงานฉบับส่งและสไลด์ (PDF) (#15) techineethamoonree authored yesterday - ✓ 5 / 5 Verified c9fe1ef <>
Commits on Oct 3, 2026	เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG (#14) jakrapatiam-create authored 2 days ago - ✓ 5 / 5 Verified e67c59 <> เพิ่มสคริปต์ reset ระบบ, .env.example และเช็คสัปดาห์สาคิล (#11) super-SuPer-dev authored 2 days ago - ✓ 5 / 5 Verified 7fd0c96 <> เพิ่ม rollback DAG: ย้อน champion และสั่ง API reload จากหน้าเว็บ Airflow (#10) super-SuPer-dev authored 2 days ago - ✓ 5 / 5 Verified 709c9b1 <> เพิ่มเทสต์ monitoring/data_prep และแสดง coverage ใน CI (#13) Tammy-AP authored 3 days ago - ✓ 5 / 5 Verified e2ae73c <> เพิ่มสคริปต์รัน test case ผ่าน API สำหรับวันนำเสนอ (#12) naphat67 authored 3 days ago - ✓ 5 / 5 Verified e43e418 <>
Commits on Oct 1, 2026	fix: keep AUTO_APPROVE reachable at high thresholds: stabilize latency in model tests (#9) super-SuPer-dev authored 5 days ago - ✓ 5 / 5 Verified ebf3645 <> docs: evidence screenshots, contribution guide and PR template (#2) super-SuPer-dev authored 5 days ago - ✓ 5 / 5 Verified fb65c1b <> Loan approval MLOps pipeline: Airflow, MLflow, Flask API, monitoring, CI super-SuPer-dev committed 5 days ago - ✓ 5 / 5 a6e8264 <>

รูปที่ 17 ประวัติ commit บน main จากสมาชิกแต่ละคน

9.5 เทสต์และ coverage

ชุดเทสต์มีทั้งหมด 38 ข้อ: 30 ข้อรันใน job code-quality (data 6, model 4, API 5, monitoring 6, data_prep 3, สคริปต์ test case 4 และรายงาน Evidently 2) และอีก 8 ข้อใน tests/test_dags.py ซึ่งต้องมี Airflow จีรันใน job dag-tests และถูกข้ามอัตโนมัติบนเครื่องที่ไม่มี Airflow เมื่อรันทั้งชุดบนเครื่องที่ไม่มี Airflow ผ่าน 30 ข้อ (ข้าม 1 โฟล์)

coverage ของ src/ รวม 85% (494 statements พลาด 73) โดย config.py, features.py และ monitoring.py ได้ 100%, drift_report.py และ validation.py ได้ 94% และ steps.py ได้ 89%

9.6 สคริปต์ test case ของอาจารย์

scripts/run_test_cases.py รันไฟล์ CSV ผ่าน API จริงได้ทั้งกรณีปกติและกรณีผิดปกติ (วิธีใช้อยู่ใน docs/test_cases.md) โดยทำ 3 อย่าง

1. ตรวจสอบไฟล์กับ schema แล้วบอกว่าคอลัมน์ไหนผิดปกติอย่างไร
2. ส่งทีละแถวเข้า /predict ได้ 200 พร้อมผลตัดสิน AUTO_APPROVE / AUTO_REJECT / MANUAL_REVIEW หรือได้ 400 พร้อมเหตุผล ซึ่งเป็นพฤติกรรมที่ถูกต้องสำหรับข้อมูลเสีย
3. ถ้าไฟล์มีคอลัมน์ Loan_Approved จะคำนวณ accuracy และ ROC-AUC ให้

ผลทั้งหมดเขียนลง reports/test_cases.json และมีเทสของสคริปต์นี้ 4 ข้อ

10. การใช้ AI ช่วยงาน ข้อจำกัด และงานในอนาคต

10.1 การใช้ AI ช่วยงาน

กลุ่มใช้ Claude ช่วยเขียนโค้ดและร่างรายงานนี้ สมาชิกตรวจ รัน และอธิบายโค้ดทุกบรรทัดที่ส่งได้ตามที่โจทย์กำหนด

ตารางที่ 17 ส่วนที่ใช้ AI ช่วยและสิ่งที่สมาชิกตรวจหรือตัดสินใจเอง

ส่วนของงาน	AI ช่วยอะไร	สมาชิกตรวจ / ตัดสินใจอะไร
Notebook ต้นแบบ	ร่างโค้ดและคำอธิบายทุกชิ้น	ตรวจการตัดคอลัมน์ที่รั่วคำตอบ (Risk_Score, Approved_Amount) และกำหนดอัตราส่วนต้นทุน 3 : 1
src/, DAG, Flask API	เขียนโค้ดโครงสร้าง MLOps ต่อยอดจาก Lab 9 และ 11	ปรับ threshold และ cascade, ปรับ gating metric และสร้าง Rollback DAG บน Airflow
Docker, Prometheus, Grafana, CI	ร่างไฟล์ config และช่วยแก้ปัญหา port ขนกันบน Windows	กำหนดกฎแจ้งเตือน 7 ข้อ ออกแบบ Grafana dashboard 18 แผง และเพิ่ม unit test จน coverage ใน CI ถึง 85%
รายงานและเอกสาร	ร่างเนื้อหาจากผลที่รันจริง	ตรวจความถูกต้องของตัวเลข สรุปผลการทดสอบ และตรวจข้อความทั้งฉบับ

10.2 ข้อจำกัด

- ข้อมูลเป็นข้อมูลสังเคราะห์และไม่มีคอลัมน์เวลา drift ทั้งหมดจึงเป็นการจำลอง และในการจำลองเผลยมาพร้อม batch ทันที ยังไม่ได้จำลองความล่าช้าของเผลยที่เกิดขึ้นในงานจริง
- label คือผลการตัดสินของธนาคารในอดีต ไม่ใช่ผลการชำระหนี้ โมเดลจึงเลียนแบบการตัดสินเดิม รวมถึงอคติที่อาจมีอยู่
- ค่าขอที่ระบบตัดสินเอง (ประมาณ 78%) ไม่ผ่านเจ้าหน้าที่ จึงไม่มีเผลยตามมา ถ้าใช้เฉพาะเผลยจากโซน MANUAL_REVIEW ในการตรวจ concept drift จะเกิด selection bias
- หลังเปลี่ยนนโยบาย โมเดลใหม่เทรนจาก 1,500 แถว และ test split มีแค่ 225 แถว ตัวเลขและผล gate จึงกว้างได้มากกว่าโมเดลแรก
- concept drift ที่จำลองเป็นกฎตายตัว (เลิกปล่อยกู้เกิน 120 เดือน) ในงานจริง นโยบายที่ชัดเจนแบบนี้ควรเขียนเป็นกฎธุรกิจคู่กับโมเดล ส่วนการ retrain เหมาะกับการเปลี่ยนแปลงที่เขียนเป็นกฎไม่ได้
- โซน cascade กว้างคงที่ ± 0.10 รอบ threshold ทำให้ช่วงความน่าจะเป็น 0.50–0.62 ถูกปฏิเสธอัตโนมัติ
- ตรวจ fairness เฉพาะเพศ ยังไม่ได้ตรวจ Marital_Status และตัวแปรที่อาจเป็นตัวแทนของกลุ่มอ่อนไหว

- Flask dev server ทำงานโปรเซสเดียว ความจุสูงสุดประมาณ 75 คำขอ/วินาที
- log ของ API เก็บ request ทั้งหมด ซึ่งมีข้อมูลส่วนบุคคล
- การแจ้งเตือนจาก pipeline ส่งเข้าไฟล์และ webhook (Discord) ได้ แต่ alert ของ Prometheus ยังไม่ส่งออกไปช่องทางใด เพราะยังไม่มี Alertmanager

10.3 งานในอนาคต

- ประเมินผลโมเดลก่อน deploy มาถึงด้วย NannyML CBPE
- สุ่มคำขอที่ระบบตัดสินใจส่วนหนึ่ง (เช่น 5%) ให้เจ้าหน้าที่ตรวจสอบ เพื่อให้มีโมเดลที่ไม่ลำเอียง
- Canary release: ส่ง traffic 10% ไปที่ challenger ก่อน promote เต็ม
- เลือกความกว้างโชน cascade จากต้นทุนบน validation และรายงานความแม่นยำแยกโชน
- เปลี่ยน Flask dev server เป็น gunicorn หลาย worker
- mask ข้อมูลส่วนบุคคลใน log ก่อนเก็บ ตาม พ.ร.บ. คุ้มครองข้อมูลส่วนบุคคล (PDPA)
- เพิ่ม Alertmanager ส่ง alert ของ Prometheus เข้า Discord
- ให้เหตุผลรายคำขอ (reason codes) จาก coefficient ส่งกลับไปพร้อมผลลัพธ์

เอกสารอ้างอิง

1. Loan Approval Dataset 2026. Kaggle (สัญญาอนุญาต MIT).
2. MLflow Documentation. <https://mlflow.org/docs/latest/>
3. Apache Airflow Documentation. <https://airflow.apache.org/docs/>
4. scikit-learn User Guide. https://scikit-learn.org/stable/user_guide.html
5. Evidently Documentation. <https://docs.evidentlyai.com/>
6. Prometheus Documentation. <https://prometheus.io/docs/>
7. Grafana Documentation. <https://grafana.com/docs/>
8. เอกสารประกอบการสอนและ Lab รายวิชา CP413008 Machine Learning Engineering for Production